

Lab 7: Project setup

Q1. Why do we build on OpenZeppelin instead of writing ERC-20 from scratch?

We use OpenZeppelin because its contracts are audited, battle-tested, widely used, and already reviewed for common security and gas issues. Reusing them reduces the amount of custom code, therefore reducing the chance of introducing bugs.

```
npm audit fix --force

Run `npm audit` for details.
PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> npx hardhat --version
2.29.1
PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> npx hardhat
888 888 888 888 888
888 888 888 888 888
888 888 888 888 888
88888888888 8888b. 888d888 .d88888 88888b. 8888b. 888888
888 888 "88b 888P" d88" 888 888 "88b "88b 888
888 888 .d8888888 888 888 888 888 .d8888888 888
888 888 888 888 888 Y88b 888 888 888 888 Y88b.
888 888 "Y888888 888 "Y88888 888 888 "Y888888 "Y888

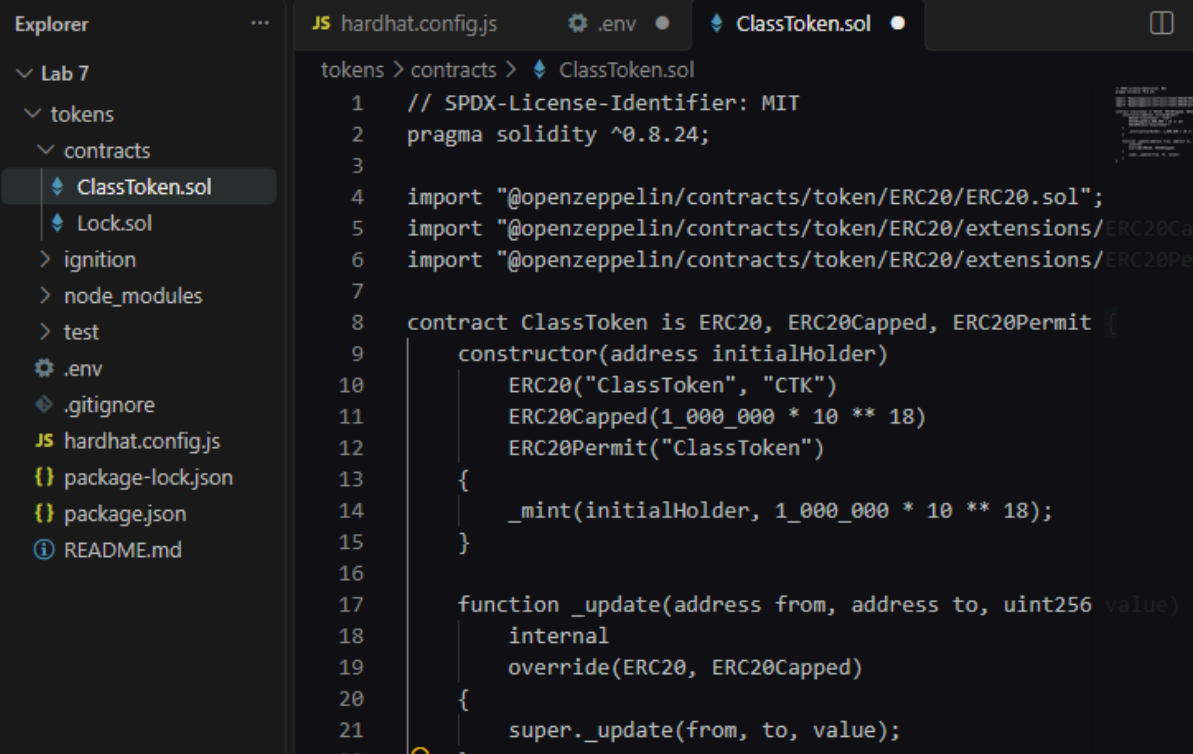
Welcome to Hardhat v2.29.1

✓ What do you want to do? · Create a JavaScript project
✓ Hardhat project root: · D:\SCHOOLWORK\Blockchain\Lab 7\tokens
? Do you want to add a .gitignore? (Y/n) » y
```

```
... JS hardhat.config.js X
tokens > JS hardhat.config.js > ...
1 require("@nomicfoundation/hardhat-toolbox");
2 require("dotenv").config();
3
4 module.exports = {
5   solidity: {
6     version: "0.8.24",
7     settings: {
8       optimizer: {
9         enabled: true,
10        runs: 200,
11      },
12      evmVersion: "paris",
13    },
14  },
15
16  networks: {
17    trustkeys: {
18      url: "https://l1testnet.trustkeys.network",
19      chainId: 11968,
20      accounts: process.env.PRIVATE_KEY
21        ? [process.env.PRIVATE_KEY]
22        : [],
23    },
24  },
25  };

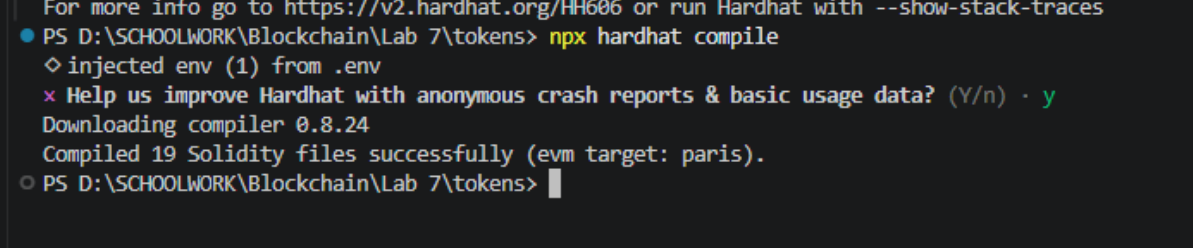
```

Lab 7.1 ClassToken



```
tokens > contracts > ClassToken.sol
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.24;
3
4 import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
5 import "@openzeppelin/contracts/token/ERC20/extensions/ERC20Capped.sol";
6 import "@openzeppelin/contracts/token/ERC20/extensions/ERC20Permit.sol";
7
8 contract ClassToken is ERC20, ERC20Capped, ERC20Permit {
9     constructor(address initialHolder)
10         ERC20("ClassToken", "CTK")
11         ERC20Capped(1_000_000 * 10 ** 18)
12         ERC20Permit("ClassToken")
13     {
14         _mint(initialHolder, 1_000_000 * 10 ** 18);
15     }
16
17     function _update(address from, address to, uint256 value)
18         internal
19         override(ERC20, ERC20Capped)
20     {
21         super._update(from, to, value);
22     }
23 }
```

.env file includes private key, not shown for privacy.



```
For more info go to https://v2.hardhat.org/HH606 or run Hardhat with --show-stack-traces
PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> npx hardhat compile
  injected env (1) from .env
  x Help us improve Hardhat with anonymous crash reports & basic usage data? (Y/n) · y
  Downloading compiler 0.8.24
  Compiled 19 Solidity files successfully (evm target: paris).
PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens>
```

Q2: decimals() returns 18. If Alice's balance is 1000000000000000000, how many CTK is that, and where does the "18" actually live — on-chain or in the UI? That balance represents 1 CTK. The 18 is display metadata used by wallets and UIs to decide where to place the decimal point; on-chain, balances are stored as integer base units.

Q3: Why does a DEX need approve + transferFrom instead of a plain transfer? A plain transfer only sends tokens directly and does not let the DEX contract pull tokens from the user's balance inside its own transaction. approve gives the DEX an allowance, and transferFrom lets it move tokens up to that allowance. An infinite approval such as $2^{256} - 1$ is risky because if the spender contract is hacked or malicious, it may drain the user's entire token balance without requiring a new approval.

```
ardhat.config.js x .env ClassToken.sol JS ClassToken.test.js
tokens > test > JS ClassToken.test.js > ...
1  const { expect } = require("chai");
2  const { ethers } = require("hardhat");
3
4  describe("ClassToken", function () {
5    let token;
6    let owner, alice, bob, spender;
7
8    const CAP = ethers.parseEther("1000000");
9
10   beforeEach(async function () {
11     [owner, alice, bob, spender] = await ethers.getSigners();
12
13     const ClassToken = await ethers.getContractFactory("ClassToken");
14     token = await ClassToken.deploy(owner.address);
15     await token.waitForDeployment();
16   });
17
18   it("has correct name, symbol, decimals, cap and total supply", function () {
19     expect(await token.name()).to.equal("ClassToken");
20     expect(await token.symbol()).to.equal("CTK");
21     expect(await token.decimals()).to.equal(18);
22
23     expect(await token.cap()).to.equal(CAP);
24     expect(await token.totalSupply()).to.equal(CAP);
25     expect(await token.balanceOf(owner.address)).to.equal(CAP);
26   });
27
28   it("transfers tokens and emits Transfer", async function () {
29     const amount = ethers.parseEther("100");
30
31     await expect(token.transfer(alice.address, amount))
32       .to.emit(token, "Transfer")
33       .withArgs(owner.address, alice.address, amount);
34   });
35 });
```

```
Compiled 19 Solidity files successfully (evm target: paris).
PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> npx hardhat test
◊ injected env (1) from .env
x Help us improve Hardhat with anonymous crash reports & basic usage data?
Y/n) · y

ClassToken
  ✓ has correct name, symbol, decimals, cap and total supply (166ms)
  ✓ transfers tokens and emits Transfer
  ✓ reverts when balance is insufficient (68ms)
  ✓ approve + transferFrom works and allowance decreases
  ✓ keeps totalSupply constant after multiple transfers

5 passing (3s)

PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> 
```

Lab 7.2 ClassBadge

```
.env X ClassToken.sol JS ClassToken.test.js ClassBadge.sol X
tokens > contracts > ClassBadge.sol
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.24;
3
4 import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
5 import "@openzeppelin/contracts/access/Ownable.sol";
6 import "@openzeppelin/contracts/utils/Base64.sol";
7 import "@openzeppelin/contracts/utils/Strings.sol";
8
9 contract ClassBadge is ERC721, Ownable {
10     using Strings for uint256;
11
12     uint256 private _nextId;
13     mapping(uint256 => string) private _studentName;
14
15     error EmptyName();
16
17     constructor(address initialOwner)
18         ERC721("ClassBadge", "BADGE")
19         Ownable(initialOwner)
20     {}
21
22     function mint(address to, string memory studentName)
23         external
24         onlyOwner
25         returns (uint256)
26     {
27         if (bytes(studentName).length == 0) {
28             revert EmptyName();
29         }
30
31         uint256 tokenId = _nextId++;
32         _studentName[tokenId] = studentName;
```

```
5 passing (3s)
PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> npx hardhat compile
◇ injected env (1) from .env
✗ Help us improve Hardhat with anonymous crash reports & basic usage data? (Y/n) · y

Compiled 9 Solidity files successfully (evm target: paris).
PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens>
```

```
.env ClassToken.sol JS ClassToken.test.js ClassBadge.sol JS ClassBadge.test.js X
t > JS ClassBadge.test.js > describe("ClassBadge") callback > it("decodes on-chain tokenURI metadata correctly") calll
1  const { expect } = require("chai");
2  const { ethers } = require("hardhat");
3
4  describe("ClassBadge", function () {
5    let badge;
6    let owner, alice;
7
8    beforeEach(async function () {
9      [owner, alice] = await ethers.getSigners();
10
11      const ClassBadge = await ethers.getContractFactory("ClassBadge");
12      badge = await ClassBadge.deploy(owner.address);
13      await badge.waitForDeployment();
14    });
15
16    it("mints a badge and stores ownership", async function () {
17      await badge.mint(alice.address, "Alice");
18
19      expect(await badge.ownerOf(0)).to.equal(alice.address);
20      expect(await badge.totalMinted()).to.equal(1);
21    });
22
23    it("only owner can mint", async function () {
24      await expect(
25        badge.connect(alice).mint(alice.address, "Alice")
26      ).to.be.revertedWithCustomError(badge, "OwnableUnauthorizedAccount");
27    });
28
29    it("reverts on empty student name", async function () {
30      await expect(
31        badge.mint(alice.address, "")
32      ).to.be.revertedWithCustomError(badge, "EmptyName");
33    });
34
35    it("reverts for an unminted token id", async function () {
```

Q4: Where do the JSON and the image live? What did we avoid by NOT using IPFS on TrustKeys?

Both the JSON metadata and the SVG image are encoded and stored on-chain through the contract's tokenURI. By not using IPFS, we avoided depending on an external pinning service or gateway to keep the metadata available.

Q5: What does the "safe" in `_safeMint` / `safeTransferFrom` actually check, and why?

If the recipient is a contract, the safe operation checks whether that contract implements `onERC721Received`. This prevents the NFT from being permanently stuck in a contract that cannot properly receive or manage ERC-721 tokens.

```

PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> npx hardhat test
◇ injected env (1) from .env
× Help us improve Hardhat with anonymous crash reports & basic usage data? (Y/n) · y

ClassBadge
  ✓ mints a badge and stores ownership
  ✓ only owner can mint
  ✓ reverts on empty student name
  ✓ reverts for an unminted token id
  ✓ decodes on-chain tokenURI metadata correctly

ClassToken
  ✓ has correct name, symbol, decimals, cap and total supply
  ✓ transfers tokens and emits Transfer
  ✓ reverts when balance is insufficient
  ✓ approve + transferFrom works and allowance decreases
  ✓ keeps totalSupply constant after multiple transfers

10 passing (627ms)
PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens>

```

Lab 7.3 Deploy & interact on TrustKeys

```

tokens > scripts > JS deploy-token.js > ...
1  const { ethers } = require("hardhat");
2
3  async function main() {
4      const [deployer] = await ethers.getSigners();
5
6      console.log("Deploying ClassToken with account:", deployer.
7
8      const ClassToken = await ethers.getContractFactory("ClassTo
9
10     const token = await ClassToken.deploy(deployer.address);
11     await token.waitForDeployment();
12
13     console.log("ClassToken deployed to:", await token.getAddre
14 }
15
16 main().catch((error) => {
17     console.error(error);
18     process.exitCode = 1;
19 });

```

```
tokens > scripts > JS deploy-badge.js > ...
1  const { ethers } = require("hardhat");
2
3  async function main() {
4      const [deployer] = await ethers.getSigners();
5
6      console.log("Deploying ClassBadge with account:", deployer
7
8      const ClassBadge = await ethers.getContractFactory("ClassB
9
10     const badge = await ClassBadge.deploy(deployer.address);
11     await badge.waitForDeployment();
12
13     console.log("ClassBadge deployed to:", await badge.getAddr
14 }
15
16 main().catch((error) => {
17     console.error(error);
18     process.exitCode = 1;
19 });
```

```
10 passing (627ms)

● PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> npx hardhat run scripts/deploy-token.js --network trustkeys
◇ injected env (1) from .env
✖ Help us improve Hardhat with anonymous crash reports & basic usage data? (Y/n) · y
◇ injected env (0) from .env
Deploying ClassToken with account: 0x3D0785ce2a09c6aFb0fd3947E0cdC6bC9e4745d9
ClassToken deployed to: 0x8b44c4f56c9a0F16Df344E21F2DB2972B3f52ED2
○ PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> █
```

```
● PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> npx hardhat run scripts/deploy-badge.js --network trustkeys
◇ injected env (1) from .env
✖ Help us improve Hardhat with anonymous crash reports & basic usage data? (Y/n) · y
◇ injected env (0) from .env
Deploying ClassBadge with account: 0x3D0785ce2a09c6aFb0fd3947E0cdC6bC9e4745d9
ClassBadge deployed to: 0x25fBB307A9bb19757eB5647517e28328D09502a4
○ PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> █
```

Q6: TrustKeys has no public explorer. How did you confirm the 100-CTK transfer landed?
We confirmed the transfer using ethers view calls such as balanceOf before and after the transaction, and we could also verify the updated token balance in MetaMask. In our output, the sender balance decreased from 1,000,000 CTK to 999,900 CTK, while the recipient balance increased from 0 CTK to 100 CTK.

```
tokens > scripts > JS interact-token.js > ...
1  const { ethers } = require("hardhat");
2
3  async function main() {
4      const tokenAddress = process.env.TOKEN;
5      const to = process.env.TO;
6
7      if (!tokenAddress || !to) {
8          throw new Error("Please set TOKEN and TO environment variables");
9      }
10
11     const [sender] = await ethers.getSigners();
12
13     console.log("Sender:", sender.address);
14     console.log("Token:", tokenAddress);
15     console.log("Recipient:", to);
16
17     const token = await ethers.getContractAt("ClassToken", tokenAddress);
18
19     const amount = ethers.parseEther("100");
20
21     const beforeSender = await token.balanceOf(sender.address);
22     const beforeRecipient = await token.balanceOf(to);
23
24     console.log(
25         "Sender balance before:",
26         ethers.formatEther(beforeSender),
27         "CTK"
28     );
29
30     console.log(
31         "Recipient balance before:",
32         ethers.formatEther(beforeRecipient),
33         "CTK"
34     );
35 }
```

```
PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> npx hardhat run scripts/interact-token.js --network trustkeys
• injected env (1) from .env $env:TOKEN="0x8b44c4f56c9a0f16df344e21f2db2972b3f52ed2"
>> $env:TO="0x9f6ed78099f6c6822d9b885770cd3239df5da56"& basic usage data? (Y/n) · y
>> injected env (0) from .env
• >> npx hardhat run scripts/interact-token.js --network trustkeys
  injected env (1) from .env
  x Help us improve Hardhat with anonymous crash reports & basic usage data? (Y/n) · y
  injected env (0) from .env
  Sender: 0x3D0785ce2a09c6aFb0fd3947E0cdC6bC9e4745d9
  Token: 0x8b44c4f56c9a0f16df344e21f2db2972b3f52ed2
  Recipient: 0x9f6ed78099f6c6822d9b885770cd3239df5da56
  Sender balance before: 1000000.0 CTK
  Recipient balance before: 0.0 CTK
  Transaction hash: 0xa9d57b190ff80ac40b3ea8fc6c3d10678e90dfe1f9910efe83eb4283fc1b2403
  Sender balance after: 999900.0 CTK
  Recipient balance after: 100.0 CTK
PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> 
```

Gửi 100 CTK tới tài khoản Bùi Quang Minh

```
en.js JS deploy-badge.js JS interact-token.js JS mint-badge.js
tokens > scripts > JS mint-badge.js > main > tx
1  const { ethers } = require("hardhat");
2
3  async function main() {
4      const badgeAddress = process.env.REGISTRY;
5
6      if (!badgeAddress) {
7          throw new Error("Please set REGISTRY environment variable")
8      }
9
10     const [owner] = await ethers.getSigners();
11
12     console.log("Owner:", owner.address);
13     console.log("ClassBadge:", badgeAddress);
14
15     const badge = await ethers.getContractAt(
16         "ClassBadge",
17         badgeAddress
18     );
19
20     const tx = await badge.mint(
21         owner.address,
22         "Nam Tran"
23     );
24
25     console.log("Transaction hash:", tx.hash);
26
27     await tx.wait();
28
29     console.log("Badge #0 minted successfully");
30     console.log("Owner of token #0:", await badge.ownerOf(0));
31 }
32
33 main().catch((error) => {
34     console.error(error);
35     process.exitCode = 1;
36 });
```

```
● PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> $env:REGISTRY="0x25fbb307a9bb19757eb5647517e28328d09502a4"
● PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> npx hardhat run scripts/mint-badge.js --network trustkeys
◇ injected env (1) from .env
× Help us improve Hardhat with anonymous crash reports & basic usage data? (Y/n) · y
◇ injected env (0) from .env
Owner: 0x3D0785ce2a09c6aFb0fd3947E0cdC6bC9e4745d9
ClassBadge: 0x25fbb307a9bb19757eb5647517e28328d09502a4
Transaction hash: 0xf7ed199c905041321a1d159b71598ed3576cd77f46a9ef9f0155f21569f2c72f
Badge #0 minted successfully
Owner of token #0: 0x3D0785ce2a09c6aFb0fd3947E0cdC6bC9e4745d9
○ PS D:\SCHOOLWORK\Blockchain\Lab 7\tokens> 
```


Q7: What stops someone from replaying your permit signature on a second chain or a second time?

The permit signature is bound to the specific chainId, contract address, owner nonce, and deadline. The nonce prevents the same permit from being reused, while the chainId and verifying contract prevent replay on another chain or another contract; the deadline also limits how long the signature remains valid.